

08-vue-router: vue路由管理器

一、介绍

1. 什么是路由？

1.1 生活中的路由

路由器的本质就是 ip 与 设备的映射关系



1.2 Vue中的路由

路径和页面的映射关系

<https://bitejiuyeye.com/index>





1.3 介绍vue-router

Vue Router | Vue.js 的官方路由

Vue 官方的一个路由插件, 是一个第三方模块包

Vue Router Vue.js 的官方路由

为 Vue.js 提供富有表现力、可配置的、方便的路由

入门→

🎥 免费视频课程

Get the Vue Router Cheat Sheet

📖 富有表现力的路由语法

用直观且强大的语法来定义静态或动态路由。

● 细致的导航控制

可拦截任何导航并更精确地控制其结果。

📦 基于组件的配置方法

将每条路由映射到应该显示的组件上。

📜 支持历史模式

有 HTML5、hash 或记忆历史模式可供选择。

📖 支持滚动控制

可精确控制每个页面的滚动位置。

🔗 支持自动编码

可直接在代码中使用 unicode 字符 (你好)。

1.4 总结

1. 什么是路由?

答: 一种映射(对应)关系

2. Vue中的路由是什么?

答: 路径和页面的映射关系

3. Vue中如何实现路由切换?

答: 借助 **vue-router** 模块

4. vue-router是什么?

答: Vue的官方路由管理器

2. 单页应用程序介绍

2.1 概念

单页应用程序，简称SPA(Single Page Application)，指的是所有的功能都在一个HTML网页中实现

2.2 具体示例

单页应用网站：比如：比特就业课 <https://bitejiuyeye.com/index>，网易云音乐 <https://music.163.com/>

多页应用网站：比如：京东 <https://jd.com/>， 淘宝 <https://taobao.com>

2.3 单页面 VS 多页面

开发方式	简称	实现方式	页面性能	开发效率	用户体验	首屏加载	SEO
单页面	SPA	一个HTML页面	按需更新 性能高	高	好	较慢	差
多页面	MPA	多个HTML页面	整体更新 性能低	中等	一般	快	优

2.4 总结

1. 什么是单页面应用程序？

答：所有的功能都在一个html页面上实现

2. 单页面应用优缺点？

答：1、优点：体验好、开发效率高

2、缺点：首屏加载相对较慢、不利于SEO

3. vue文件的分类

3.1 为什么分类

人为的把 .vue 文件分为两类，仅仅是为了便于理解和管理，但二者本质无区别

- 页面组件：独立展示的，配合路由切换，不可复用的
- 复用组件：用于组装页面组件，可复用的

<https://bitejiuyeye.com/index>

<https://juejin.cn/>

3.2 存放目录

1. 页面组件 - 页面展示 - 配合路由使用

放置在 `src/views` 目录下

2. 复用组件 - 用于组装页面组件

放置在 `src/components` 目录下

3.3 总结

1. 组件分为哪两类？分类的目的？

答：页面组件、复用组件；便于管理

2. 不同分类的组件应该放在什么文件夹？作用分别是什么？

答：1、 页面组件 -> `src/views` -> 配合路由切换

2、 复用组件 -> `src/components` -> 组装页面组件

二、基本使用和模块封装

1. 基本使用(4+2)

1.1 四个固定步骤

如下4个固定的步骤（不用背）

1. 下载 VueRouter 模块

```
1 yarn add vue-router
```

2. 导入相关函数

```
1 import { createRouter, createWebHashHistory } from 'vue-router'
```

3. 创建路由实例

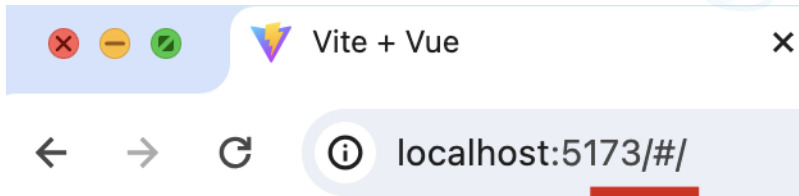
```
1 const router = createRouter({  
2   // 哈希模式，路径带 #  
3   history: createWebHashHistory(),  
4   routes: [  
5     // 路由表规则，即 path 与 component 的对应关系  
6   ]  
7 })
```

4. 注册，将路由实例注册到应用中，让规则生效

```
1 app.use(router)
```

当完以上 4 步之后, 就可以看到浏览器地址栏中的路径变成了 `/#/` 的形式。

表示项目的路由已经被 Vue-Router 管理了



1.2 两个核心步骤

1. views目录下, 创建需要的页面组件, 并配置路由规则



views/Find.vue

```
1 <script setup></script>
2
3 <template>
4   <div class="find">
5     <p>发现音乐</p>
6     <p>发现音乐</p>
7     <p>发现音乐...</p>
8   </div>
9 </template>
10
11 <style scoped></style>
```

views/My.vue

```
1 <script setup></script>
2
3 <template>
4   <div class="my">
```

```
5     <p>我的音乐</p>
6     <p>我的音乐</p>
7     <p>我的音乐...</p>
8   </div>
9 </template>
10
11 <style scoped></style>
```

views/Friend.vue

```
1  <script setup></script>
2
3  <template>
4    <div class="friend">
5      <p>朋友</p>
6      <p>朋友</p>
7      <p>朋友...</p>
8    </div>
9  </template>
10
11 <style scoped></style>
```

main.js

```
1  // 导入两个相关函数
2  // createRouter(): 创建路由实例
3  // createWebHashHistory(): 创建哈希模式的路由, 路径带 #
4  import { createRouter, createWebHashHistory } from 'vue-router'
5
6  // 导入 3 个页面组件
7  import Find from '@views/Find.vue'
8  import Friend from '@views/Friend.vue'
9  import My from '@views/My.vue'
10
11 // 创建路由实例
12 const router = createRouter({
13   history: createWebHashHistory(),
14   routes: [
15     {
16       path: '/find',
17       component: Find
18     }, {
19       path: '/my',
```

```

20     component: My
21   }, {
22     path: '/friend',
23     component: Friend
24   }
25 ]
26 })
27
28 // 注册
29 app.use(router)

```

2. 给路由出口(路径匹配的组件, 显示的位置)

App.vue

```

1  <script setup></script>
2
3  <template>
4    <!-- 路由出口 -->
5    <router-view />
6  </template>

```

1.3 路由运作原理

当浏览器url改变时, 匹配路由表数组中的path值, 如果命中了, 则把相应的component渲染到 `<router-view />` 的位置; 否则显示空白

1.4 总结

1. 如何实现 路径改变, 对应组件 切换, 应该使用哪个插件?

答: `vue-router`

2. vue-router的使用步骤是什么(4+2)?

答: 下载->导入->创建->注册-> `配置规则表`->给出口

2. 抽离封装路由模块

2.1 问题

路由配置代码都写在 main.js 中合适吗? 显然不合适, 会让 main.js 代码变得臃肿

2.2 目标

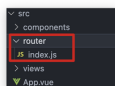
将路由模块抽离出来。好处: **利于管理和维护**

```
import {
  createRouter,
  createWebHashHistory
} from 'vue-router' // 31.7k (gziped: 12k)

import Find from './views/Find.vue'
import Friend from './views/Friend.vue'
import My from './views/My.vue'

const router = createRouter({
  history: createWebHashHistory(),
  routes: [
    {
      path: '/find',
      component: Find
    },
    {
      path: '/my',
      component: My
    },
    {
      path: '/friend',
      component: Friend
    }
  ]
})
```

抽离到一个
单独模块中



导出
导入

```
// main.js
src > # main.js > ...
1 /** @format */
2
3 import { createApp } from 'vue'
4 import App from './App.vue'
5
6 import router from './router'
7
8 const app = createApp(App)
9
10 app.use(router)
11
12 app.mount('#app')
```

2.3 代码示例

新建 router/index.js

```
1 import { createRouter, createWebHashHistory } from 'vue-router'
2
3 // 注意：脚手架环境下 @ 代指 src 目录，可以用于快速引入组件
4 import Find from '@views/Find.vue'
5 import Friend from '@views/Friend.vue'
6 import My from '@views/My.vue'
7
8 const router = createRouter({
9   history: createWebHashHistory(),
10   routes: [
11     {
12       path: '/find',
13       component: Find
14     },
15     {
16       path: '/my',
17       component: My
18     },
19     {
20       path: '/friend',
21       component: Friend
22     }
23   ]
24 })
25
26 // 导出路由实例
27 export default router
```

main.js

```
1 import { createApp } from 'vue'
2 import App from './App.vue'
3
4 // 导入路由实例
```



```
5 import router from './router'
6
7 const app = createApp(App)
8
9 // 注册
10 app.use(router)
11
12 app.mount('#app')
```

2.4 总结

1. 封装抽离路由模块的好处是什么？

答：便于管理、维护

2. 以后如何快速引入组件？

答：可以借助 @ 代指 src 目录

三、声明式导航与传参

1. 声明式导航

1.1 问题

要手动输入地址，切换页面，不合理吧？能否点击链接进行跳转

1.2 解决方案

vue-router 提供了一个全局组件 router-link, 用于点击跳转，需添加 to 属性指定路径，其本质还是 a 标签

语法：<router-link to="path值"> xxx </router-link>

```
1 <script setup></script>
2
3 <template>
4   <nav>
5     <router-link to="/find">发现音乐</router-link>
6     <router-link to="/my">我的音乐</router-link>
7     <router-link to="/friend">朋友</router-link>
8   </nav>
9   <!-- 一级路由出口 -->
10  <router-view />
11 </template>
12
13 <style>
```

```

14     nav a {
15         color: #333;
16         text-decoration: none;
17     }
18     nav a:nth-child(2) {
19         margin: 0 80px;
20     }
21     nav a.router-link-active {
22         background: red;
23         color: #fff;
24     }
25 </style>

```

1.3 自带高亮类名

使用router-link跳转后，我们发现。当前点击的链接默认加了两个class的值

`router-link-exact-active` 和 `router-link-active`，

我们可以给任意一个class属性添加高亮样式即可实现功能

1.4 总结

1. router-link是什么？

答：声明式导航，点击跳转路由的

2. router-link怎么用？

答：添加 `to` 属性

3. router-link的好处是什么？

答：自带 激活类名，方便实现高亮样式

2. 两个类名

当我们使用跳转时，自动给当前导航加了两个类名

```

<nav> == $0
<a href="#/find" class="router-link-active router-link-exact-active" aria-current="page">发现音乐</a>
<a href="#/my" class>我的音乐</a>
<a href="#/friend" class>朋友</a>
</nav>

```

2.1 router-link-active

模糊匹配

只要是以/find开头的路径都可以和 `to="/find"` 匹配到

2.2 router-link-exact-active

精确匹配

to="/find" 仅可以匹配 /find

2.3 总结

1. router-link 会自动给当前导航添加两个类名，有什么区别呢？

答：1、router-link-active：模糊匹配

2、router-link-exact-active：精确匹配

3. 声明式导航-传查询参

3.1 目标

在跳转路由时，进行传参，比如：现在我们在发现音乐页，点击去朋友页，并携带id，方便后续查询详情，如何传参？

3.2 跳转传参

我们可以通过两种方式，在跳转的时候把所需要的参数传到其他页面中

1. 查询参数传参

2. 动态路由传参

3.3 查询参数传参

- 传参(有2种格式)

a. 字符串

```
1 <router-link to="/path?参数名=值"> xxx </router-link>
```

a. 对象

```
1 <router-link :to="{
2   path: '/path',
3   query: {
4     参数名: 值
5     ...
6   }
7 }"> xxx </router-link>
```

- 接收参数

```
1 <script setup>
```

```

2   import { useRoute } from 'vue-router'
3   // 得到当前激活的路由对象
4   const route = useRoute()
5   // 获取查询参数
6   console.log(route.query)
7 </script>

```

3.4 代码示例

App.vue

```

1  <!-- 字符串 -->
2  <router-link to="/friend?id=10086"> 朋友 </router-link>
3
4  <!-- 对象 -->
5  <router-link :to="{
6    path: '/friend',
7    query: {
8      id: 10086
9    }
10 }"> 朋友 </router-link>

```

Friend.vue

```

1  <script setup>
2    import { useRoute } from 'vue-router'
3    const route = useRoute()
4    console.log(route.query.id)
5  </script>

```

3.5 总结

1. 路由查询参数如何传递？

答：字符串："/path?参数名=值" 或 对象：path+query

2. 查询参如何接收？

答：route.query.参数名

4. 声明式导航-传动态参

4.1 动态路由传参方式

- 配置动态路由

- 动态路由后面的参数可以随便起名，但要有语义

```
const router = createRouter({
  history: createWebHashHistory(),
  routes: [
    { ... },
    { ... },
    {
      path: '/friend/:fid',
      component: Friend
    }
  ]
})
```

```
1  const router = createRouter({
2    ...
3    routes: [
4      ...
5      {
6        path: '/friend/:fid',
7        component: Friend
8      }
9    ]
10  })
```

- 传递参数(有2种格式)

a. 字符串

```
1  <router-link to="/path/具体值"> xxx </router-link>
```

a. 对象

```
1  <router-link :to="{
2    name: 'Friend',
3    params: {
4      参数名: 具体值
5    }
6  }"> xxx </router-link>
```

- 接收参数

```
1  <script setup>
```

```
2   import { useRoute } from 'vue-router'
3   const route = useRoute()
4   // 获取动态路由参数
5   console.log(route.params)
6 </script>
```

4.2 代码示例

router/index.js

```
1  createRouter({
2    routes: [
3      {
4        name: 'Friend',
5        path: '/friend/:fid',
6        component: Friend
7      }
8    ]
9  })
```

App.vue

```
1  <!-- 字符串 -->
2  <router-link to="/friend/10010"> 朋友 </router-link>
3
4  <!-- 对象 -->
5  <router-link :to="{
6    name: 'Friend',
7    params: {
8      fid: 10010
9    }
10 }"> 朋友 </router-link>
```

Friend.vue

```
1  <script setup>
2    import { useRoute } from 'vue-router'
3    const route = useRoute()
4    console.log(route.params.fid)
5  </script>
```

4.3 总结

1. 路由动态参数如何传递？

答：1、 `/:参数名` 先占位

2、 字符串: `"/path/具体值"` 或 对象: `name+params`

2. 查询参如何接收？

答: `route.params.参数名`

5. 查询参 VS 动态参

5.1 对比

1. 查询参数传参 (比较适合传多个参数)

a. 跳转: `to="/path?参数名=值&参数名2=值"`

b. 获取: `route.query.参数名`

2. 动态路由传参 (优雅简洁, 传单个参数比较方便)

a. 配置动态路由: `path: "/path/:参数名"`

b. 跳转: `to="/path/参数值"`

c. 获取: `route.params.参数名`

3. 注意: 动态路由也可以传多个参数, 但一般只传一个

5.2 总结

1. 声明式导航跳转时, 有几种方式传值给路由页面？

答: 2种, `查询参数传参` 和 `动态路由传参`

四、更多配置

1. 重定向

1.1 问题

网页打开时, url 默认是 / 路径, 未匹配到组件时, 会出现空白



1.2 解决方案

重定向: 匹配 / 后, 比如强制跳转 /find 路径, 避免页面空白

1.3 语法

```
1 { path: 匹配路径, redirect: 要重定向的路径 }
```

1.4 代码示例

```
1 const router = createRouter({
2   ...
3
4   routes: [
5     // 访问 / , 自动跳转到 /find
6     { path: '/', redirect: '/find' },
7     ...
8   ]
9 })
```

2. 404

2.1 作用

当路径找不到匹配时, 给个提示页面

2.2 位置

404的路由, 虽然配置在任何一个位置都可以, 但一般都配置在其他路由规则的最后面

2.3 语法

path: "*" (任意路径) – 前面不匹配就命中最后这个

```
1 import _404 from '@views/404.vue'
2
3 const router = new VueRouter({
4   routes: [
5     ...
6     { path: '*', component: _404 } // 404配置, 推荐放在路由表的最后一个
7   ]
8 })
```

2.4 代码

views/404.vue

```
1 <script setup> </script>
2
3 <template>
4   <div>
5     <h3>404</h3>
6     <p> 你访问的页面去了月球 </p>
7     <router-link to="/"> 去首页 </router-link>
8   </div>
9 </template>
```

router/index.js

```
1 ...
2 import _404 from '@views/404'
3
4 const router = createRouter({
5   routes: [
6     ...
7
8     { path: '*', component: _404 }
9   ]
10 })
11
12 export default router
```

3. 模式

3.1 问题

路由的路径看起来不好看，有#，能否切成真正路径形式？

- hash路由(默认) 例如: <http://localhost:5173/#/find>
- history路由(常用) 例如: <http://localhost:8080/find> (上线需要服务器端支持，开发环境Vite给规避掉了history模式的问题)

3.2 语法

```
1 const router = new VueRouter({
2   // 历史模式: createWebHistory(), 路径带 #
3   // 哈希模式: createWebHashHistory(), 路径不带 #
```

```
4     history: createWebHashHistory(),
5
6     routes: []
7   })
```

五、编程式导航与传参

1. 编程式导航

1.1 问题

如何主动做路由跳转？比如：登录成功自动跳转至首页

1.2 解决方案

编程式导航：用JS代码来进行跳转

1.3 语法

路由实例 `router.push(路径)`

```
1  <script setup>
2    import { useRouter } from 'vue-router'
3    const router = useRouter()
4    router.push(字符串)
5    router.push(对象)
6  </script>
```

1.4 代码示例

Find.vue

```
1  <script setup>
2    import { useRouter } from 'vue-router'
3    import { onMounted } from 'vue'
4    const router = useRouter()
5    // 组件挂载后，延迟2秒，自动从 发现音乐页 跳转至 朋友页
6    onMounted(() => {
7      setTimeout(() => {
8        router.push('/friend')
9
10       router.push({
11         path: '/friend'
12       })
13     })
14   })
```

```
13
14     router.push({
15         name: 'Friend'
16     })
17 }, 2000)
18 })
19 </script>
```

1.5 总结

1. 程式化导航如何做路由跳转？

答： `router.push(字符串/对象)`

2. 程式化导航传参

2.1 问题

程式化导航如何传参呢？

2.2 语法

与 声明式导航 `<router-link>` 传参和接参方式完全一样，既支持字符串，也支持对象

```
1  router.push('/path?参数名=值')
2
3  router.push({
4      path: '/path',
5      query: {
6          参数名: 值
7      },
8      ...
9  })
10
11 router.push({
12     name: '路由名称',
13     params: {
14         参数名: 值
15     }
16 })
```

2.3 代码示例

2.3.1 查询参数

router/index.js

```

1  createRouter({
2    routes: [
3      { path: '/friend', component: Friend }
4    ]
5  })

```

Find.vue

```

1  router.push('/friend?fid=110')
2
3  router.push({
4    path: '/friend',
5    query: {
6      fid: 101
7    }
8  })

```

Friend.vue

```

1  <script setup>
2    import { useRoute } from 'vue-router'
3
4    const route = useRoute()
5    console.log(route.query.fid)
6  </script>

```

2.3.2 动态参数

router/index.js

```

1  createRouter({
2    routes: [{
3      // 命名路由
4      name: 'Friend',
5      path: '/friend/:id',
6      component: Friend
7    }]
8  })

```

Find.vue

```

1  router.push('/friend/110')
2
3  router.push({
4    name: 'Friend'
5    params: {
6      id: 101
7    }
8  })

```

Friend.vue

```

1  <script setup>
2    import { useRoute } from 'vue-router'
3    const route = useRoute()
4    console.log(route.params.id)
5  </script>

```

2.4 总结

1. 程式化导航如何传参和接参？

答：同声明式导航完全一样，`router.push(字符串/对象)`

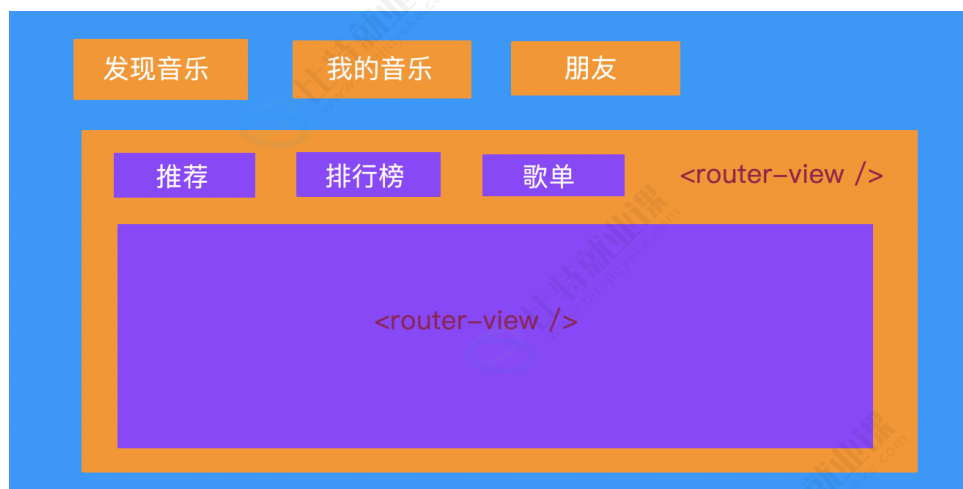
六、嵌套与守卫

1. 嵌套

1.1 问题

能否在一个路由页面中，再进行一套路由的切换呢？比如 [网易云音乐](#) 的发现音乐页

1.2 效果图



1.3 步骤

1. 创建3个二级路由页面组件(Recommend、Ranking、SongList)
2. 路由表数组中, 在相应的一级路由规则中, 配置 `children` 规则
3. 在相应的一级路由页面组件中, 给二级路由 `<router-view/>` 出口和导航链接

注意: 1、二级路由的 path 推荐 不加 `/`

2、做路由跳转的时候要写 完整路径(父路径+当前路径)

3、防止二级路由页面空白, 给默认显示的一级路由 添加重定向

1.4 代码示例

views/Recommend.vue

```
1  <script setup></script>
2
3  <template>
4    <div class="recommend">
5      <h5>推荐</h5>
6      <h5>推荐</h5>
7      <h5>推荐</h5>
8    </div>
9  </template>
10
11 <style scoped></style>
```

views/Ranking.vue

```
1  <script setup></script>
2
3  <template>
4    <div class="ranking">
5      <h5>排行榜</h5>
6      <h5>排行榜</h5>
7      <h5>排行榜</h5>
8    </div>
9  </template>
10
11 <style scoped></style>
```

views/SongList.vue

```
1 <script setup></script>
2
3 <template>
4   <div class="songlist">
5     <h5>歌单</h5>
6     <h5>歌单</h5>
7     <h5>歌单</h5>
8   </div>
9 </template>
10
11 <style scoped></style>
```

router/index.js

```
1 import Recommend from '@/views/Recommend.vue'
2 import Ranking from '@/views/Ranking.vue'
3 import SongList from '@/views/SongList.vue'
4
5 const router = createRouter({
6   history: createWebHashHistory(),
7   routes: [{
8     path: '/find',
9     component: Find,
10    // 重定向, 防止二级路由空白
11    redirect: '/find/recommend',
12    // 配置嵌套路由
13    children: [{
14      // 二级路由开始, 推荐 path 不加 /
15      path: 'recommend',
16      component: Recommend
17    }, {
18      path: 'ranking',
19      component: Ranking
20    }, {
21      path: 'songlist',
22      component: SongList
23    }
24  ]
25 })
```

views/Find.vue

```

1  <script setup></script>
2
3  <template>
4    <div class="find">
5      <nav>
6        <!-- 二级路由导航链接 -->
7        <router-link to="/find/recommend">推荐</router-link>
8        <router-link to="/find/ranking">排行榜</router-link>
9        <router-link to="/find/songlist">歌单</router-link>
10     </nav>
11     <!-- 二级路由出口 -->
12     <router-view />
13   </div>
14 </template>
15
16 <style scoped></style>
17

```

1.5 总结

1. 如何配置路由嵌套？

答：配 `children`，给路由 出口`<router-view/>`

2. 需要注意什么？

答：二级 path 不加 /；跳转要写完整路径；避免空白、要添加重定向

2. 路由守卫

2.1 问题

能否在访问某个路由前, 添加权限判断? 比如 我的音乐页, 只有登录了才可以访问

2.2 解决方案

路由(导航)全局前置守卫: 每个路由在跳转前都会触发回调函数

```

1  router.beforeEach((to, from) => {
2    // to: 即将要进入的路由
3    // from: 当前正要离开的路由
4
5    // false 取消导航(不发生跳转)
6    return false
7    // 正常放行、正常跳转
8    return undefined | true
9    // 重定向到指定的路由
10   return '/路径'

```



```
11  })
```

2.3 代码示例

```
1  // 用来模拟是否登录
2  const isLogin = true
3
4  router.beforeEach((to, from) => {
5    // 如果没有登录, 并且还要去 我的音乐页
6    if(!isLogin && to.path === '/my') {
7      // 进行提示
8      alert('请先登录')
9      // 不放行(不跳转)
10     return false
11   }
12   // 正常放行
13   return true
14 })
```

2.4 总结

1. 如何在访问路由前添加权限校验?

答: 全局前置守卫 `router.beforeEach((to, from) => { })`

2. 参数 to、from 和 回调函数返回值 都表示什么?

- 1 to: 即将进入的路由
- 2 from: 正要离开的路由
- 3 回调函数返回值:
- 4 return false: 不放行
- 5 return true/undefined: 放行
- 6 return 路径: 重定向到指定路由

